When approaching this scenario, it is important to strike a balance among all of the involved stakeholders. The code needs to be easy to parse for backend developers while also maintaining usability for the end-user (customers). The modularity of the code should result in more resource efficiency and faster computations. For an e-commerce platform, where monetary transactions are involved, speed and accuracy are of the utmost importance. To achieve these goals, we can use reusable functions, array manipulation, and various string functions.

The string functions strtoupper and strtolower (Bro Code, 2024) proved most useful for parts 1 and 3, where I had to capitalize the first word of a product name and print an entire product description in lower-case characters, respectively. The "." used for concatenation (PHP Group, 2025b) was crucial in all three parts for maintaining eligibility and allowing the program to flow nicely. As for array functions, the array_merge function (PHP Group, 2025a) was very useful for part 2c. Through the use of the aforementioned techniques, the program has a logical flow to it that any programmer on the team can comprehend and is functional for the customer.

The comments in the code further explain my reasoning for specific aspects.

1.

Code screenshot:

```php
<?php
// 1a. Creates a function to calculate the price of a customer's order including a 10% sales tax
function calculateTotal($price, $quantity) { // Defines function name with price and quantity parameters
    $subtotal = $price * $quantity; // Subtotal,tax, and total variables are calculated through arithmetic.
    $tax = $subtotal * 0.10;
    $total = $subtotal + $tax;
    return $total;
}

// 1b. This function formats the product name by eliminating extra spaces, capitalizing the first letter of each word, and limiting it to 50 characters.
function formatProductName($name) {
    // Trim and collapse multiple spaces
    $name = trim($name);
    $name = preg_replace('/\s+/', ' ', $name);

    // Title-case each word (multibyte-safe for non-latin scripts)
    if (function_exists('mb_convert_case')) {
        $name = mb_convert_case(mb_strtolower($name, 'UTF-8'), MB_CASE_TITLE, 'UTF-8');
    } else {
        $name = ucwords(strtolower($name));
    }

    // Establishes a 50 character maximum limit.
    if ((function_exists('mb_strlen') ? mb_strlen($name, 'UTF-8') : strlen($name)) <= 50) {
        return $name;
    }

    // Ensures the 50 character limit by truncating characters if the maximum limit is exceeded
    if (function_exists('mb_substr') && function_exists('mb_strrpos')) {
        $cut = mb_substr($name, 0, 50, 'UTF-8');
        $lastSpace = mb_strrpos($cut, ' ', 0, 'UTF-8');
        if ($lastSpace !== false) {
            return rtrim(mb_substr($cut, 0, $lastSpace, 'UTF-8'));
        }
        return rtrim($cut);
    } else {
        $cut = substr($name, 0, 50);
        $lastSpace = strrpos($cut, ' ');
        return $lastSpace !== false ? rtrim(substr($cut, 0, $lastSpace)) : rtrim($cut);
    }

}

// 1c. Function that applies a discount.
function calculateDiscount($price, $discountPercent) {
    $discount = $price * ($discountPercent / 100); // Arithmetic to calculate the discounted price.
    return $price - $discount;
}

// Output tests
echo "Total: 2 (quanitity) x 1000 (price): " .calculateTotal(2, 1000). "<br>";
echo "Formatted name: " .formatProductName("Huawei pure 80 ultra"). "<br>";
echo "Discounted price: 1000 (price) x 25% (discount): " . calculateDiscount(1000, 25). "<br>";
?>
```

Code text:

<?php

// 1a. Creates a function to calculate the price of a customer's order including a 10% sales tax

function calculateTotal($price, $quantity) { // Defines function name with price and quantity parameters

    $subtotal = $price * $quantity; // Subtotal,tax, and total variables are calculated through arithmetic.

```
    $tax = $subtotal * 0.10;
    $total = $subtotal + $tax;
    return $total;
}

// 1b. This function formats the product name by eliminating extra spaces, capitalizing the first letter of each word, and limiting it to 50 characters.
function formatProductName($name) {
        // Trim and collapse multiple spaces
    $name = trim($name);
    $name = preg_replace('/\s+/', ' ', $name);

    // Title-case each word (multibyte-safe for non-latin scripts)
    if (function_exists('mb_convert_case')) {
        $name = mb_convert_case(mb_strtolower($name, 'UTF-8'), MB_CASE_TITLE, 'UTF-8');
    } else {
        $name = ucwords(strtolower($name));
    }

    // Establishes a 50 character maximum limit.
    if ((function_exists('mb_strlen') ? mb_strlen($name, 'UTF-8') : strlen($name)) <= 50) {
        return $name;
    }
```

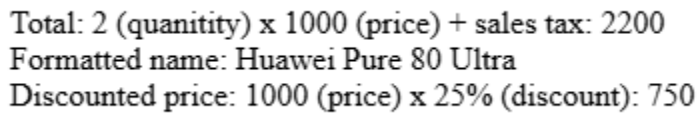
```
    // Ensures the 50 character limit by truncating characters if the maximum limit is exceeded
    if (function_exists('mb_substr') && function_exists('mb_strrpos')) {
        $cut = mb_substr($name, 0, 50, 'UTF-8');
        $lastSpace = mb_strrpos($cut, ' ', 0, 'UTF-8');
        if ($lastSpace !== false) {
            return rtrim(mb_substr($cut, 0, $lastSpace, 'UTF-8'));
        }
        return rtrim($cut);
    } else {
        $cut = substr($name, 0, 50);
        $lastSpace = strrpos($cut, ' ');
        return $lastSpace !== false ? rtrim(substr($cut, 0, $lastSpace)) : rtrim($cut);
    }

}

// 1c. Function that applies a discount.
function calculateDiscount($price, $discountPercent) {
    $discount = $price * ($discountPercent / 100); // Arithmetic to calculate the discounted price.
    return $price - $discount;
}
```

```
// Output tests
echo "Total: 2 (quanitity) x 1000 (price) + sales tax: " .calculateTotal(2, 1000). "<br>";
echo "Formatted name: " .formatProductName("Huawei pure 80 ultra"). "<br>";
echo "Discounted price: 1000 (price) x 25% (discount): " . calculateDiscount(1000, 25). "<br>";
?>
```

Output screenshot:



2.

Code screenshot:

```php
<?php
// 2a. Stores an array of products including the name, price, and category in the $products variable
$products = [
    ["name" => "Lenovo Laptop", "price" => 1500, "category" => "Electronics"],
    ["name" => "Sony Headphones", "price" => 100, "category" => "Electronics"],
    ["name" => "English Novel", "price" => 20, "category" => "Book"],
    ["name" => "Lenovo Laptop", "price" => 1500, "category" => "Electronics"] // Duplicate item
];

// Removes duplicate products from the array.
$products = array_unique($products, SORT_REGULAR);

// Sorts by price in ascending order.
usort($products, function($a, $b) {
    return $a['price'] <=> $b['price'];
});

// Displays list under a header.
echo "<h3>Product List (Sorted by ascending price)</h3>";
foreach ($products as $p) {
    echo $p['name'] . " - $" . $p['price'] . " (" . $p['category'] . ")<br>";
}

// 2b. Applies a 10% discount to Electronics
function calculateDiscount($price, $discountPercent) { // Function for calculating the discount
    return $price * (1 - ($discountPercent / 100));
}

echo "<h3>Electronics with 10% Discount</h3>";
foreach ($products as $p) {
    if ($p['category'] === "Electronics") {
        $discountedPrice = calculateDiscount($p['price'], 10); // Calculates discount
        echo $p['name'] . " - $" . $discountedPrice . " (" . $p['category'] . ")<br>"; // Prints updated list
    }
}

// 2c. Mergees supplier inventories and accounts for duplicates
$supplier1 = [
    ["name" => "Tablet", "price" => 300],
    ["name" => "Laptop Charger", "price" => 30]
];
$supplier2 = [
    ["name" => "Laptop Charger", "price" => 30], // Duplicate item
    ["name" => "Mechanical Keyboard", "price" => 50]
];

$merged = array_merge($supplier1, $supplier2);
$merged = array_unique($merged, SORT_REGULAR);

echo "<h3>Combined Supplier Inventory</h3>"; // Combined inventory header
foreach ($merged as $item) {
    echo $item['name'] . " - $" . $item['price'] . "<br>"; // Prints the combined supplier inventory under this header
}
?>
```

Code text:

<?php

// 2a. Stores an array of products including the name, price, and category in the $products variable

$products = [

["name" => "Lenovo Laptop", "price" => 1500, "category" => "Electronics"],

["name" => "Sony Headphones", "price" => 100, "category" => "Electronics"],

```
    ["name" => "English Novel", "price" => 20, "category" => "Book"],
    ["name" => "Lenovo Laptop", "price" => 1500, "category" => "Electronics"] // Duplicate item
];

// Removes duplicate products from the array.
$products = array_unique($products, SORT_REGULAR);

// Sorts by price in ascending order.
usort($products, function($a, $b) {
    return $a['price'] <=> $b['price'];
});

// Displays list under a header.
echo "<h3>Product List (Sorted by ascending price)</h3>";
foreach ($products as $p) {
    echo $p['name'] . " - $" . $p['price'] . " (" . $p['category'] . ")<br>";
}

// 2b. Applies a 10% discount to Electronics
function calculateDiscount($price, $discountPercent) { // Function for calculating the discount
    return $price * (1 - ($discountPercent / 100));
}
```

```
echo "<h3>Electronics with 10% Discount</h3>";
foreach ($products as $p) {
    if ($p['category'] === "Electronics") {
        $discountedPrice = calculateDiscount($p['price'], 10); // Calculates discount
        echo $p['name'] . " - $" . $discountedPrice . " (" . $p['category'] . ")<br>"; // Prints updated
list
    }
}

// 2c. Merges supplier inventories and accounts for duplicates
$supplier1 = [
    ["name" => "Tablet", "price" => 300],
    ["name" => "Laptop Charger", "price" => 30]
];
$supplier2 = [
    ["name" => "Laptop Charger", "price" => 30], // Duplicate item
    ["name" => "Mechanical Keyboard", "price" => 50]
];

$merged = array_merge($supplier1, $supplier2);
$merged = array_unique($merged, SORT_REGULAR);

echo "<h3>Combined Supplier Inventory</h3>"; // Combined inventory header
```

```
foreach ($merged as $item) {
    echo $item['name'] . " - $" . $item['price'] . "<br>"; // Prints the combined supplier inventory under this header
}
?>
```

Output screenshot:



3.

Code screenshot:

```php
<?php
// 3a. Function to format product descriptions by making it all lower-case, removing underscores, and removing special characters.
function formatDescription($desc) {
    $desc = strtolower($desc);
    $desc = str_replace("_", " ", $desc);
    $desc = preg_replace("/[^a-z0-9\s]/", "", $desc); // remove special chars
    return $desc;
}

// Test output
echo "<h3>Formatted Item Description</h3>"; // Header for organization
echo formatDescription("nEWLY released_Item!") . "<br>"; // Unformatted description as a parameter.

// 3b. Analyzes the description's constitution
function analyzeDescription($desc) {
    $charCount = strlen($desc); // Character count variable
    $wordCount = str_word_count($desc); // Word count variable
    echo "Character count: $charCount<br>";
    echo "Word count: $wordCount<br>";
    if (stripos($desc, "wallet") !== false) {
        echo "Keyword match found<br>";
    } else {
        echo "Keyword match not found<br>";
    }
}

// Test output
echo "<h3>Description Analysis</h3>"; // Header for organization.
analyzeDescription("This is a high-quality leather wallet with RFID protection."); // Product description as input parameter

// 3c. Function for customer review format analysis
function processReview($review) {
    $preview = substr($review, 0, 20) . "..."; // Preview of the first 20 characters
    $pos = stripos($review, "excellent"); // Finds the first occurence of the word "excellent."
    if ($pos !== false) {
        echo "Found 'excellent' at $pos<br>";
    }
    $updated = $review . " Thank you for your feedback!"; // Concatenates the "thank you" message with the review.
    echo "Preview: $preview<br>";
    echo "Updated review: $updated<br>";
}

// Test output
echo "<h3>Customer Review</h3>"; //header for organization
processReview("This product is great! It arrived quickly and in excellent condition."); // Customer review as the parameter.
?>
```

Code text:

<?php

// 3a. Function to format product descriptions by making it all lower-case, removing underscores, and removing special characters.

function formatDescription($desc) {

    $desc = strtolower($desc);

    $desc = str_replace("_", " ", $desc);

    $desc = preg_replace("/[^a-z0-9\s]/", "", $desc); // remove special chars

    return $desc;

}

```
// Test output
echo "<h3>Formatted Item Description</h3>"; // Header for organization
echo formatDescription("nEWLY released_Item!") . "<br>"; // Unformatted description as a parameter.

// 3b. Analyzes the description's constitution
function analyzeDescription($desc) {
    $charCount = strlen($desc); // Character count variable
    $wordCount = str_word_count($desc); // Word count variable
    echo "Character count: $charCount<br>";
    echo "Word count: $wordCount<br>";
    if (stripos($desc, "wallet") !== false) {
        echo "Keyword match found<br>";
    } else {
        echo "Keyword match not found<br>";
    }
}

// Test output
echo "<h3>Description Analysis</h3>"; // Header for organization.
analyzeDescription("This is a high-quality leather wallet with RFID protection."); // Product description as input parameter
```

```
// 3c. Function for customer review format analysis
function processReview($review) {
    $preview = substr($review, 0, 20) . "..."; // Preview of the first 20 characters
    $pos = stripos($review, "excellent"); // Finds the first occurence of the word "excellent."
    if ($pos !== false) {
        echo "Found 'excellent' at $pos<br>";
    }
    $updated = $review . " Thank you for your feedback!"; // Concatenates the "thank you"
message with the review.
    echo "Preview: $preview<br>";
    echo "Updated review: $updated<br>";
}

// Test output
echo "<h3>Customer Review</h3>"; //header for organization
processReview("This product is great! It arrived quickly and in excellent condition."); //
Customer review as the parameter.
?>
```

Output screenshot:

Formatted Item Description

newly released item

Description Analysis

Character count: 59
Word count: 9
Keyword match found

Customer Review

Found 'excellent' at 49
Preview: This product is grea...
Updated review: This product is great! It arrived quickly and in excellent condition. Thank you for your feedback!

References:

Bro Code. (2024, May 20). *PHP string functions are awesome* [Video]. YouTube.

https://youtu.be/B-neQLCsORM

PHP Group. (2025a). Array operators. *PHP Manual*. Licensed Under Creative Commons

Attribution 3.0. https://www.php.net/manual/en/language.operators.array.php

PHP Group. (2025b). String Operators. *PHP Manual*. Licensed Under Creative Commons

Attribution 3.0. https://www.php.net/manual/en/language.operators.string.php